



Databases and Applications

Unit III

Pooja T S

Computer Applications

Databases and Applications

Unit III

Indexing: Single, Compound, Text in MongoDB

Pooja T S

Computer Applications



Databases and Applications

MongoDB Indexing – Introduction

► What is an Index?

A data structure that stores keys in a **sorted** manner.

Helps MongoDB locate documents **without scanning entire collections**.

Based on a **B-tree model**, enabling fast lookups and range queries.

► Why Indexes Matter?

Improve read performance significantly.

Reduce query latency for filters and sorting.

Essential for large datasets and real-time applications.



Databases and Applications

How MongoDB Uses Indexes?

► Execution Flow

Query Planner checks available indexes.

Chooses the **best index** using cost-based optimisation.

Avoids collection scan when a matching index exists.

► Index Structure

Uses a **B-tree** to store ordered key-value references.

Supports equality, range, prefix and sort queries.

Allows compound indexes to cover multiple fields.



Databases and Applications

Creating Indexes

► Single-Field Index

Syntax: `db.collection.createIndex({ field: 1 })`

1 = ascending, -1 = descending

Improves queries like: `find({ field: value })`

► Compound Index

Created as: `db.collection.createIndex({ a:1, b:1 })`

Order matters for query planning.

Supports prefix-based lookups.



Databases and Applications

Index Types in MongoDB

▶ Common Index Types

Single Field Index – best for equality lookups.

Compound Index – multi-column search.

Multikey Index – for array fields.

▶ Specialised Index Types

Text Index – for search on textual content.

Hashed Index – uniform hashing for sharding.

Geospatial Index – for location-based queries.



Databases and Applications

Index Properties in MongoDB

► Unique Indexes

Enforces uniqueness of field values.

Syntax: `createIndex({email:1}, {unique:true})`

Used for usernames, emails, IDs.

► Sparse + TTL Indexes

Sparse: Includes only documents with indexed field.

TTL: Auto-deletes documents after given seconds.

Syntax: `expireAfterSeconds:3600`



Databases and Applications

Index Builds in MongoDB

► **Foreground Build**

Blocks writes but is fastest.

Suitable for offline maintenance.

Not recommended for production workloads.

► **Background Build**

Allows reads/writes during build.

Slightly slower but safer in production.

Use: `{ background: true }`



Databases and Applications

Managing Indexes

► View Existing Indexes

```
db.collection.getIndexes()
```

Shows fields, properties, and index name.

Helps verify accidental large indexes.

► Dropping Indexes

Drop one: `db.collection.dropIndex("name")`

Drop all: `db.collection.dropIndexes()`

Avoid excessive index count to save memory.



Databases and Applications

Measuring Index Performance

► Using explain()

`explain("executionStats")` reveals plan used.
Checks if index prevents COLLSCAN.
Helps diagnose slow queries.

► Key Metrics

Index hit ratio.
Keys examined vs documents examined.
Query execution time (ms).



Databases and Applications

Indexing Strategy – Best Practices

► What to Index

Fields used in frequent queries.

Fields used in sorting operations.

Foreign-key-like fields (customerId etc.).

► What Not to Index

High-cardinality fields rarely queried.

Booleans (true/false) – low selectivity.

Large multi-key arrays – expensive to maintain.



Databases and Applications

Text Indexes – Concept

- ▶ What is a text index?
 - Special index type for full-text search on string content
 - Tokenises text into words and indexes them
 - Used with `$text` queries
- ▶ Rules and limits
 - At most **one** text index per collection
 - Can include multiple string fields in the same text index
 - Does not support partial word match (no prefix search like `"dev*"`)



Databases and Applications

Text Indexes – Example

- ▶ Blog posts collection: posts
 - Fields: title, body, tags
 - Need to search posts by keywords in title or body
 - Use a combined text index
- ▶ Shell commands
 - `db.posts.createIndex({ title:"text", body:"text" })`
 - Search: `db.posts.find({ $text:{ $search:"mongodb index" } })`
 - Relevance score: `project { score:{ $meta:"textScore" } }`



Databases and Applications

Indexing – Putting It All Together

► Choosing index types

- Single field: simple filters or sort on one field
- Compound: frequent multi-field filters and sorts (respect prefix rule)
- Text: keyword search on large string fields (`$text` queries)

► Good practices

- Index only fields that appear in `find()`, `sort()`, or join-like patterns
- Avoid too many indexes on write-heavy collections
- Use `db.collection.explain()` to confirm index usage



Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 391